

THE LAYOUT CATALOGUE

Every value, in one place. Keep this one open while you work — the lecture deck is the story, this is the lookup.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN

display — WHAT KIND OF BOX

VALUE	LINE BEHAVIOUR	WIDTH / HEIGHT	VERTICAL MARGIN
<code>block</code>	starts a new line, takes the width offered	yes	yes
<code>inline</code>	flows in the line, content width	ignored	ignored
<code>inline-block</code>	flows in the line	yes	yes
<code>flex</code>	block-level container, children become flex items	yes	yes
<code>inline-flex</code>	inline-level container, children still flex items	yes	yes
<code>none</code>	no box at all	—	—

`inline-block` is an *inline* box, so the whitespace between two of them in the source is rendered as a space. That is the gap.

LOOK AT THE BOXES THE BROWSER MADE

demos/01-normal-flow — no layout CSS at allreloadopen

☒ outline every box ☐ show the line boxes ☐ show the containing block

■ block box

■ inline box

■ line box

■ containing block

Normal flow

This paragraph is a **block box**. It takes the whole width it is offered, and the next block box starts underneath it. Nobody asked for that: it is what the browser does when you say **nothing**.

Inline content flows

Inside a block box, text and **inline boxes** are poured into **line boxes** — horizontal strips as wide as the block, filled one after another until the words run out. An **inline box** is exactly as wide as its content, and an  sits on the text baseline like a very large letter.

- A list item is a block box too.
- So they stack.

Everything on this page was laid out before you wrote a single layout rule.

Hover an element in the page above.

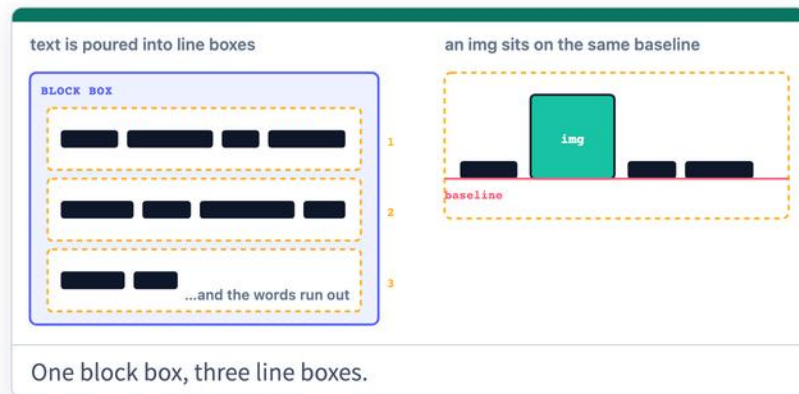
Hover any element in the page above to measure it. Nothing here is drawn by hand — the strips and the frame are positioned from `getClientRects()`.

► **HANDS-ON** `demos/01-normal-flow/` — Turn on the outlines. Blue is a block box, orange is an inline box. Then hover anything and read what it was offered.

INLINE CONTENT IS POURED, NOT STACKED

Inside a block box, text and inline boxes are poured into **line boxes** — horizontal strips as wide as the block, filled one after another until the words run out.

That is why a line-height changes the height of a paragraph, and why an `img` in a sentence sits on the text baseline like a very large letter.



SO THIS IS WHY LAST WEEK'S `width` DID NOTHING

WEEK 3, AND IT DID NOTHING

```
1 nav a {  
2   width: 160px;  
3   height: 44px;  
4 }
```

You wrote it. It was correct CSS. Nothing happened.

THE MISSING SENTENCE

```
1 nav a {  
2   display: inline-block;  
3   width: 160px;  
4   height: 44px;  
5 }
```

An inline box is sized by its content. Always was.

THE SAME THREE DECLARATIONS, THREE TIMES

demos/02-display/01-inline-wont-size [↶ reload](#) [open ↷](#)

THE SAME DECLARATIONS, THREE TIMES

display: inline
Text before the link and text after it, on the same line.

display: inline-block
Text before the link and text after it, on the same line.

display: block
Text before
the link

► **HANDS-ON** [demos/02-display/01-inline-wont-size/](#) — Ask the room for the first link's height before you read the measured number.

THREE CARDS AT 33.333% DO NOT FIT IN 100%

NOTHING IS WRONG WITH THIS CSS

```
1 .card {  
2   display: inline-block;  
3   width: 33.333%;  
4   margin: 0;  
5 }  
6  
7 /* 3 × 33.333% = 100%  
8    so why did the third  
9    one drop to a new  
10   line? */
```

THE CULPRIT IS IN THE HTML

```
1 <div class="card">one</div>  
2 <div class="card">two</div>  
3 <div class="card">three</div>  
4 <!-- the newline and the indentation  
5      between the tags are WORDS -->
```

An inline-block is an inline box. Inline boxes are separated by whitespace.

► **HANDS-ON** [demos/02-display/02-inline-block-gaps/](#) — Four fixes, and only one of them is a good idea.

FOUR WAYS TO HIDE, AND WHAT EACH ONE COSTS

METHOD	KEEPS ITS SPACE	REACHABLE BY TAB	READ BY A SCREEN READER
<code>display: none</code>	no	no	no
<code>hidden</code> attribute	no	no	no
<code>visibility: hidden</code>	yes	no	no
<code>opacity: 0</code>	yes	yes	yes

`opacity: 0` is **only transparent**. It is still there, still focusable and still clickable — which is a bug that is very hard to see.

FOUR WAYS TO MAKE SOMETHING DISAPPEAR

`display: none`

No box. No space, no tab stop, not read aloud. The honest "gone".

`visibility: hidden`

Invisible, **keeps its space**. Not focusable, not read aloud.

`opacity: 0`

Only transparent. Still takes space, still focusable, still read aloud, **still clickable**.

► **HANDS-ON** [demos/02-display/03-none-vs-hidden/](#) — Choose `opacity: 0`, then click where the button should be and watch the counter.

position — THE FIVE VALUES

VALUE	IN NORMAL FLOW	OFFSETS MEASURED FROM	ON SCROLL
<code>static</code>	yes	offsets ignored	scrolls
<code>relative</code>	yes — keeps its space	its own original position	scrolls
<code>absolute</code>	no	nearest positioned ancestor	scrolls
<code>fixed</code>	no	the viewport	stays put
<code>sticky</code>	yes — keeps its space	itself, then its scroll container	scrolls, then pins

"Out of flow" means nothing reserves its space any more: the siblings close the gap, and a parent with no other content collapses to nothing.

absolute — MEASURED FROM WHAT, EXACTLY?

- 1 **Walk up**
from the element, parent by parent.
- 2 **Stop**
at the first ancestor whose `position` is not `static`.
- 3 **Measure**
the offsets from that ancestor's padding box.
- 4 **If there is none**
, fall back to the page itself.

That is the whole algorithm. Step 4 is where everybody gets hurt, because it **always succeeds** — it never errors, it just anchors somewhere you did not mean.

```
1 .card {  
2   position: relative; /* ← the anchor */  
3 }  
4  
5 .badge {  
6   position: absolute;  
7   top: -10px;  
8   right: -10px;  
9 }
```

MISCONCEPTION 1 · THE BADGE THAT FLEW AWAY

[demos/03-position/01-absolute-adrift](#)[↶ reload](#)[open ↷](#)

☐ **position:** relative on the middle card ☐ show the ancestor chain

THE BADGE BELONGS TO THE MIDDLE CARD

the page

one

two

three

NEW

The badge landed 525px away from the card it belongs to.

```
.badge { position: absolute; top: -10px; right: -10px; }  
- unchanged. The badge never moves in this demo.
```

walking up from the badge, looking for a positioned ancestor:

```
<div#card>      static  
<div#row>       static  
<div.page-root> relative ← this one anchors the badge  
<body>         static  
<html>         static
```

The walk went straight past the card – it is `static`, so it does not count – and stopped at the page. The offsets are measured from THERE.

"If it aligns to the page instead of the box you expected, you probably forgot position: relative on the ancestor."

► **HANDS-ON** [demos/03-position/01-absolute-adrift/](#) — The badge's CSS never changes. Turn on the ancestor chain.

REBUILD IT AT HOME · THE BADGE AND THE ANCHOR

- 1 **Type the badge — absolute, negative offsets.**
Watch it fly to the corner.
- 2 **Ask for the fix before you give it.**
Let someone say "relative on the card".
- 3 **Add it, and point out that nothing else moved.**
- 4 **Then break it again — put `opacity: 0.99` on the row.**

► **SEVEN MINUTES, AT HOME.** `live-code/02-badge-and-anchor/start/` is the file; `end/` and `checkpoint.png` are what it becomes.

WHAT EACH BOX IS MEASURED AGAINST

- in normal flow** The nearest block-level ancestor's **content box**.
- absolute** The nearest ancestor with `position` other than `static` — its **padding box**.
- fixed** The viewport — *unless* an ancestor has a `transform`, `filter` or `perspective`, in which case that ancestor.
- sticky** Its nearest **scroll container**. No scroll container, no sticking.

Every percentage in an element — `width: 50%`, `left: 10%`, `padding: 5%` — resolves against this box.

Percentage heights are the exception that bites: they need the containing block to have a *definite* height. If it is `auto`, `height: 100%` resolves to `auto` too.

Use `min-height: 100dvh` instead. It asks nobody.

MISCONCEPTION 5 · height: 100% OF WHAT?

demos/05-overflow/02-height-100

↻ reload

open ↗

☐ height:100% on <html> ☐ height:100% on <body> ☐ height:100% on .wrapper instead: min-height on the box itself

THE BLUE BOX SAYS height: 100%. IT ALWAYS HAS.

```
html
  body
    .wrapper
      height: 100%
```

```
.target { height: 100% } ← never changes in this demo
```

the chain above it:

html	height: auto	0px	← auto: as tall as its children
body	height: auto	0px	← auto: as tall as its children
.wrapper	height: auto	0px	← auto: as tall as its children

► **HANDS-ON** [demos/05-overflow/02-height-100/](#) — Turn the three switches on left to right. Nothing happens until the last one.

ASK NOBODY: USE A VIEWPORT UNIT

NEEDS THREE ANCESTORS TO COOPERATE

```
1 html, body { height: 100%; }  
2 .wrapper { height: 100%; }  
3 .hero { height: 100%; }
```

NEEDS NOBODY

```
1 .hero {  
2   min-height: 100dvh;  
3 }
```

`dvh` is the *dynamic* viewport height — the one that stays correct when a phone browser slides its toolbar away.

WHAT CREATES A STACKING CONTEXT

position

Not static, **and** a z-index other than auto

position

fixed or sticky — always, z-index or not

opacity

Anything below 1

transform · filter

Anything other than none

will-change

Naming any property that would create one

isolation

isolate — the only one you write on purpose

mix-blend-mode

Anything other than normal

Inside a stacking context, z-index values are compared **only with each other**. Against anything outside, what competes is the *context's own* z-index.

So a child with z-index: 999 inside a context whose own z-index is auto loses to a sibling of that context with z-index: 1.

Raising the number never helps. Find the ancestor and either remove what made it a context, or give it a z-index.

z-index, THE MODEL EVERYONE HAS

Bigger number in front. Default is 0. Only works on positioned elements.

All three of those are true, and together they are still not enough to explain the next slide.

```
1 .dropdown {  
2   position: absolute;  
3   z-index: 999;  
4 }  
5  
6 .header {  
7   position: sticky;  
8   z-index: 1;  
9 }
```

999 > 1. So the dropdown is in front. Obviously.

MISCONCEPTION 2 · WHEN 999 LOSES TO 1

[demos/04-stacking/01-z-index-lies](#) [↶ reload](#) [open ↷](#)

dropdown z-index ☐ opacity: 0.99 on the panel ☐ transform: translateZ(0) on the panel ☐ filter: saturate(1) on the panel ☐ will-change: transform on the panel ☐ isolation: isolate on the panel

THE GREEN DROPDOWN HAS Z-INDEX 999. THE PINK HEADER HAS Z-INDEX 1.

Menu

I am z-index: 999

The dropdown is on top, as asked.

```
.panel { }           ← creates no stacking context
.dropdown { z-index: 999 }
.header { z-index: 1 }
```

Both are in the same stacking context – the root – so the two numbers are genuinely comparable, and the bigger one wins.

Now tick any box above. None of them mentions layering.

► **HANDS-ON** [demos/04-stacking/01-z-index-lies/](#) — Ask what `opacity: 0.99` will do. Then tick it. Then let someone try 99999.

overflow

VALUE	CLIPPED	SCROLL CONTAINER	CONTENT STILL REACHABLE
<code>visible</code>	no — paints outside	no	yes
<code>hidden</code>	yes	yes	yes — Tab and Ctrl-F scroll to it
<code>scroll</code>	yes	yes, bars always shown	yes
<code>auto</code>	yes	yes, bars when needed	yes
<code>clip</code>	yes	no	no

Any scroll container becomes the box a `sticky` descendant sticks inside — which is the usual reason a correct `sticky` does nothing.

REFERENCE · OVERFLOW

WHEN THE CONTENT DOES NOT FIT

visible

The default. Content paints outside the box, over whatever is under it.

hidden

Clipped — and it becomes a **scroll container**. The content is still findable by `Ctrl-F` and by `Tab`.

auto

A scrollbar only when one is needed. The one you usually want.

► **HANDS-ON** <demos/05-overflow/01-overflow-values/> — Pick `hidden`, then press `Ctrl-F` in the frame and search for **BURIED**.

A SCROLL CONTAINER IS ALSO WHAT **sticky** STICKS INSIDE

The most common reason a perfectly correct `position: sticky` does nothing is **an overflow somebody added to an ancestor for an unrelated reason**.

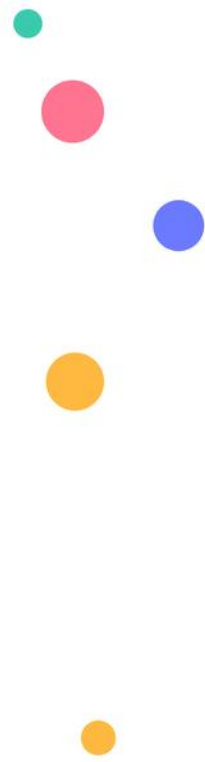
A sticky element can only travel inside its own scroll container. If that container is no taller than the element, it never has anywhere to stick.

HOW TO FIND IT

Walk up the ancestors in DevTools looking for `overflow` that is not `visible`. It is almost always three or four levels up.

FLEXBOX · ON THE CONTAINER

display	<code>flex</code> · <code>inline-flex</code>
flex-direction	<code>row</code> · <code>row-reverse</code> · <code>column</code> · <code>column-reverse</code>
flex-wrap	<code>nowrap</code> · <code>wrap</code> · <code>wrap-reverse</code>
justify-content	<code>flex-start</code> · <code>flex-end</code> · <code>center</code> · <code>space-between</code> · <code>space-around</code> · <code>space-evenly</code> — main axis
align-items	<code>stretch</code> · <code>flex-start</code> · <code>flex-end</code> · <code>center</code> · <code>baseline</code> — cross axis
align-content	Same values, but for the <i>lines</i> . Only does anything with <code>wrap</code> .
gap	<code>row-gap</code> and <code>column-gap</code> . Space <i>between</i> items only, never outside.



THE SAME NAVBAR, FOUR WAYS

[demos/06-flex/01-why-flex](#)[↶ reload](#)[open ↷](#)

[normal flow](#)[inline-block](#)[float](#)[flex](#)

THE MARKUP NEVER CHANGES. ONLY THE STYLESHEET DOES.

Ruppin FE
Home
Work
About
Sign in

This paragraph comes after the bar in the markup.

```
/* nothing at all */
```

```
bar height      187px  
links on one row no – they are on 4 rows  
"Sign in" at the end yes
```

Every `<li>` is a block box, so they stack. This is not a bug – it is the default, and it is the right default for a list.

The question the rest of act 5 answers: how do you ask for a row?

► **HANDS-ON** [demos/06-flex/01-why-flex/](#) — Run the `float` version and ask why the gold bar has no height.

REBUILD IT AT HOME · THE NAVBAR

1 Ask for the goal — brand left, links right, one line, centred.

2 Take suggestions and type them.

Wrong ones too — undo is free.

3 Land on `display: flex + align-items: center + gap`.

4 Then `margin-inline-start: auto`, and ask what it did.

► **EIGHT MINUTES, AT HOME.** `live-code/01-stack-to-row/start/` is the file; `end/` and `checkpoint.png` are what it becomes.

WHICH AXIS IS WHICH

FLEX-DIRECTION	MAIN AXIS	CROSS AXIS
<code>row</code>	horizontal →	vertical ↓
<code>row-reverse</code>	horizontal ←	vertical ↓
<code>column</code>	vertical ↓	horizontal →
<code>column-reverse</code>	vertical ↑	horizontal →

`justify-content` is **always** the main axis.

`align-items` is **always** the cross axis.

Neither of them is "horizontal". Which direction they act in is decided by `flex-direction`, one line earlier.

Do not memorise "justify is horizontal". It is true in `row` and false in `column`, and it is the single most common Flexbox mistake.

MISCONCEPTION 3 · WATCH THE AXIS MOVE

demos/06-flex/playground — every property, live reload open

THE CONTAINER

flex-direction
row

flex-wrap
nowrap

justify-content — horizontal now
flex-start

align-items — vertical now
stretch

align-content (needs wrap)
normal

gap
10px

THE SELECTED ITEM

flex-grow
0

flex-shrink

main axis →

cross axis ↓

1 2 3 — a taller item, with more words in it 4 5

```
.container {  
  display: flex;  
  gap: 10px;  
}
```

container 1278 × 240 main axis is horizontal
item 1 56 × 216 ← selected
item 2 56 × 216
item 3 190 × 216
item 4 56 × 216
item 5 56 × 216

► **HANDS-ON** [demos/06-flex/playground/](#) — Switch to `column`. The arrows turn, and the labels rewrite themselves.

FLEXBOX · ON AN ITEM

- flex-grow** Share of the *leftover* space this item takes. Default `0`.
- flex-shrink** Share of the *overflow* this item gives up. Default `1`.
- flex-basis** The size it starts from, along the main axis. Default `auto` = its content.
- align-self** Overrides `align-items` for this one item.
- order** Default `0`. Lower comes first. Negative allowed. **Visual only.**

Start with `flex-basis` to set the natural size, then tweak `flex-grow` and `flex-shrink`.



THE THREE NUMBERS

[demos/06-flex/02-grow-shrink-basis](#) [↶ reload](#) [open ↷](#)

the default (0 1 auto)

all three: flex: 1

all three: flex: auto

grow 2 / 1 / 1

a sidebar: 200px + flex: 1

item 1

flex-grow
0

flex-shrink
1

flex-basis
auto

item 2

flex-grow
0

flex-shrink
1

flex-basis
auto

item 3

flex-grow
0

flex-shrink
1

flex-basis
auto

THREE ITEMS IN A 620PX RAIL, WITH TWO 10PX GAPS

one

two

three — the item with a good deal more text in it

container	596px
– two gaps	20px
– Σ basis	403px
free space	173px

► **HANDS-ON** [demos/06-flex/02-grow-shrink-basis/](#) — The panel does the arithmetic and then prints the browser's measured widths beside it. They match.

THE SHORTHAND YOU WILL ACTUALLY READ

flex: 1 `1 1 0%` — basis zero, so content is ignored and widths come out equal.

flex: auto `1 1 auto` — basis is the content, so wider items stay wider.

flex: none `0 0 auto` — never grows, never shrinks. For the logo.

That first difference is most of the confusion about `flex` in other people's code.

```
1 .card {  
2   /* every card the same width */  
3   flex: 1;  
4 }  
5  
6 .card {  
7   /* wider cards stay wider */  
8   flex: auto;  
9 }
```

THE `flex` SHORTHAND

YOU WRITE	IT MEANS	USE IT WHEN
<code>flex: 1</code>	<code>1 1 0%</code>	equal widths, content ignored
<code>flex: auto</code>	<code>1 1 auto</code>	wider content stays wider
<code>flex: none</code>	<code>0 0 auto</code>	never grows or shrinks — a logo
<code>flex: 1 1 17rem</code>	as written	the responsive card row
<code>flex: 0 0 200px</code>	as written	a fixed sidebar

The free space is $\text{container} - \text{gaps} - \sum \text{basis}$, and each item gets $\text{basis} + (\text{free} \times \text{its grow} \div \sum \text{grow})$. When the free space is negative, `flex-shrink` takes over instead.

MISCONCEPTION 4 · `margin: auto` IS NOT A LAYOUT SYSTEM

THE HABIT

Reaching for `margin: 0 auto`, then a negative margin, then another one, until it looks right at the width you happen to be at.

THE ONE REAL USE, IN FLEX

An auto margin **absorbs the free space**. In a flex row that makes it a positioning tool:

`margin-inline-start: auto` pushes one item, and everything after it, to the end.

AND THE OTHER ONE

`margin-block-start: auto` in a flex *column* pins an item to the bottom. That is how the pricing buttons line up tonight.

THE ONE THAT WILL COST YOU AN AFTERNOON

[demos/06-flex/03-min-width-auto](#) [↶ reload](#) [open ↷](#)

☐ put a long unbreakable string in the content ☐ min-width: 0 on .content ☐ + text-overflow: ellipsis flex-shrink on .content  1

A 150PX SIDEBAR AND A flex: 1 CONTENT COLUMN

sidebar
150px

The content column is flex: 1, so it should take what is left.

The row fits its container.

```
container inner width  1498px
the flex row wants    1498px
.content min-width     auto
.content flex-shrink   1
```

Nothing is wrong yet. The content is ordinary text, which can wrap, so its minimum content width is small and flex-shrink has room to work.

Tick the first box to give it something that cannot wrap.

► **HANDS-ON** [demos/06-flex/03-min-width-auto/](#) — Turn on the long string. Now try to fix it with `flex-shrink`. Drag it to 10. Nothing.

order AND row-reverse — AND THE WARNING

Both change the **visual** order of items without touching the HTML. Both are genuinely useful, and both are a trap.

Tab order, screen-reader order and copy-paste order all follow the **DOM**. Reorder visually and you have made three orders that disagree — and only one of them is the one you tested.

Rule: if the order matters, fix the HTML. Use these for presentation, never for meaning.

align-self

Overrides `align-items` for one item.

order

Default 0. Lower comes first. Negative is allowed.

align-content

Only does anything when `flex-wrap: wrap` is on.

@media (min-width: 48em) — THE ONE YOU ARE ALLOWED

A media query asks the **viewport** a question. Flexbox cannot ask one — so the single decision this week that depends on the screen gets the single query in the file.

min-width

Mobile-first: the phone is the default, the query adds the laptop.

48em, not 768px

Relative to the browser's font size — enlarged text gets the simple layout sooner.

the two given lines

A closed `details` is zero width without its summary; its contents are skipped via `::details-content` (Baseline 2025).

```
1 @media (min-width: 48em) {  
2   .menu-toggle { display: none; }  
3  
4   /* zero width without its summary */  
5   .menu { flex: 1; }  
6  
7   /* painted whatever the open state */  
8   .menu::details-content {  
9     content-visibility: visible;  
10  }  
11  
12  .menu-list,  
13  .menu[open] .menu-list {  
14    position: static;  
15    flex-direction: row;  
16  }  
17 }
```

Week 5 is media queries properly. This week: this one, no other.

THE FIVE THAT WILL COST YOU TIME

1 · ABSOLUTE WITH NO ANCHOR

It aligned to the page, not to your box. `position: relative` is missing on the **ancestor**.

2 · Z-INDEX ACROSS CONTEXTS

999 lost to 1. An ancestor has `opacity`, `transform` or `filter`. Raising the number cannot help.

3 · JUSTIFY / ALIGN SWAPPED

You changed `flex-direction` and the axes went with it. Neither one is "horizontal".

4 · MARGIN: AUTO EVERYWHERE

It absorbs free space — that is all. Legitimate for centring a block, and for pushing one flex item to the end.

5 · HEIGHT: 100% DOING NOTHING

100% of `auto` is `auto`. Use `min-height: 100dvh`.

AND THE BONUS · MIN-WIDTH

A flex item will not shrink below its content. `min-width: 0` is the fix, and it is on the item.

WHAT `float` IS STILL FOR

Wrapping text around an image. That is what it was invented for in 1996, and it is still the only thing it does that nothing else does.

Today, using floats for overall page layout is discouraged.

If you find `float: left` on a column in a stylesheet you have inherited, you are looking at 2010. Replace it with a flex row and delete the clearfix.

```
1 .pull-quote {  
2   float: inline-start;  
3   width: 14rem;  
4   margin-inline-end: 1rem;  
5 }
```

Legitimate. Text flows around it, which is exactly the job.

WHERE TO LOOK THINGS UP

REFERENCE

MDN — search "mdn flexbox", "mdn position". The *Formal definition* table on each page tells you the default and whether it inherits.

PRACTICE

Flexbox Froggy — twenty-four levels, twenty minutes, free. Do it once and the property names stop needing to be looked up.

IN THE BROWSER

DevTools marks every flex container with a `flex` badge. Click it and the axes, the gaps and the free space are drawn on the page.

And the demos from today are yours: `demos/06-flex/playground/` answers most questions faster than a search does.